

Computational Thinking

Discrete Mathematics

Topic 03 : Collections

Number Theory

Logic

Lecture 02 : Sequence Collections

Dr Kieran Murphy 

Computing and Mathematics, SETU (Waterford).
(kieran.murphy@setu.ie)

Graphs and
Networks

Autumn Semester, 2026/27

Collections

Outline

- Mathematical concept of a sequence, AP and GP
- Sequence collections
- Lists, tuples, and strings

Enumeration

Relations & Functions

Outline

1. Sequences	2
2. Arithmetic and Geometric Progressions	16
2.1. Definition of Arithmetic and Geometric Progression	17
2.2. Partial Sums of AP and GP	19
3. Implementing Sequence Collections in Python	25
3.1. Common Concepts	26
3.2. Lists	27
3.3. Tuples	37
4. Strings	38

Sequence

Informally, a **sequence** is just an ordered list of numbers. Since the order is important we can label the values in the list, starting with zero, then one and so on. This gives us the formal definition of a sequence

Definition 1 (Sequence)

A **sequence** is a function from the set of natural numbers, $\mathbb{N} = \{0, 1, 2, 3, 4, \dots\}$ to a some set A . So we have



and

- a_n is the image of n , and is called the n^{th} **term/element** of the sequence.
- To refer to the *entire* sequence at once, we will write $(a_n)_{n \in \mathbb{N}}$ or $(a_n)_{n \geq 0}$, or if we are being sloppy, just (a_n) (in which case we assume we start the sequence with a_0).
- The numbers in the subscripts are called **indices** (the plural of **index**).

Sequence

Informally, a **sequence** is just an ordered list of numbers. Since the order is important we can label the values in the list, starting with zero, then one and so on. This gives us the formal definition of a sequence

Definition 1 (Sequence)

A **sequence** is a function from the set of natural numbers, $\mathbb{N} = \{0, 1, 2, 3, 4, \dots\}$ to a some set A . So we have



and

- a_n is the image of n , and is called the n^{th} **term/element** of the sequence.
- To refer to the *entire* sequence at once, we will write $(a_n)_{n \in \mathbb{N}}$ or $(a_n)_{n \geq 0}$, or if we are being sloppy, just (a_n) (in which case we assume we start the sequence with a_0).
- The numbers in the subscripts are called **indices** (the plural of **index**).

Sequence

Informally, a **sequence** is just an ordered list of numbers. Since the order is important we can label the values in the list, starting with zero, then one and so on. This gives us the formal definition of a sequence

Definition 1 (Sequence)

A **sequence** is a function from the set of natural numbers, $\mathbb{N} = \{0, 1, 2, 3, 4, \dots\}$ to a some set A . So we have



and

- a_n is the image of n , and is called the n^{th} **term/element** of the sequence.
- To refer to the *entire* sequence at once, we will write $(a_n)_{n \in \mathbb{N}}$ or $(a_n)_{n \geq 0}$, or if we are being sloppy, just (a_n) (in which case we assume we start the sequence with a_0).
- The numbers in the subscripts are called **indices** (the plural of **index**).

Examples of Sequences

- The sequence $a_n = n^2$, where $n = 1, 2, 3, \dots$ has elements

$$1, 4, 9, 16, 25, 36, 49, \dots$$

- The sequence $a_n = (-1)^n$, where $n = 0, 1, 2, \dots$ has elements

$$1, -1, 1, -1, 1, -1, \dots$$

- The sequence $a_n = 2^n$, where $n = 0, 1, 2, \dots$ has elements

$$1, 2, 4, 8, 16, 32, 64, 128, \dots$$

- The **Fibonacci sequence** has elements

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, \dots$$

A Quick Look at Fibonacci Sequence

During 13th century, in Liber Abaci, Fibonacci* poses the following question (paraphrasing):

Suppose we have two newly-born rabbits, one female and one male. Suppose these rabbits produce another pair of female and male rabbits after one month. These newly-born rabbits will, in turn, also mate after one month, producing another pair, and so on. Rabbits never die. How many pairs of rabbits exist after one year?

*discovered earlier by Indian scholars (Gopāla, before 1135), studying rhythmic patterns

A Quick Look at Fibonacci Sequence

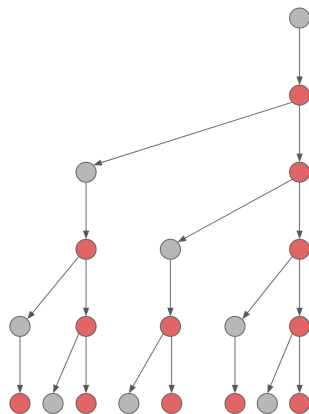
During 13th century, in Liber Abaci, Fibonacci* poses the following question (paraphrasing):

Suppose we have two newly-born rabbits, one female and one male. Suppose these rabbits produce another pair of female and male rabbits after one month. These newly-born rabbits will, in turn, also mate after one month, producing another pair; and so on. Rabbits never die. How many pairs of rabbits exist after one year?

The figure to the right illustrates this process.

- Every point denotes one rabbit pair.
- A grey point denotes a newborn pair (and not ready to reproduce).
- A red point denotes a mature, reproducing pair.

Fibonacci's Rabbits



*discovered earlier by Indian scholars (Gopāla, before 1135), studying rhythmic patterns

A Quick Look at Fibonacci Sequence

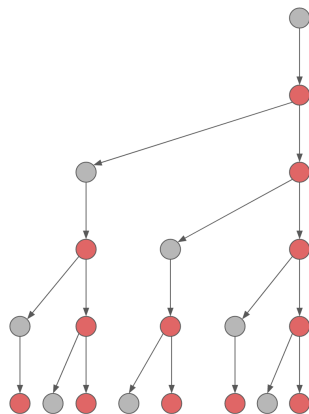
During 13th century, in Liber Abaci, Fibonacci* poses the following question (paraphrasing):

Suppose we have two newly-born rabbits, one female and one male. Suppose these rabbits produce another pair of female and male rabbits after one month. These newly-born rabbits will, in turn, also mate after one month, producing another pair; and so on. Rabbits never die. How many pairs of rabbits exist after one year?

The figure to the right illustrates this process.

- Every point denotes one rabbit pair.
- A grey point denotes a newborn pair (and not ready to reproduce).
- A red point denotes a mature, reproducing pair.

Fibonacci's Rabbits



*discovered earlier by Indian scholars (Gopāla, before 1135), studying rhythmic patterns

Suppose we have two newly-born rabbits, one female and one male. Suppose these rabbits produce another pair of female and male rabbits after one month. These newly-born rabbits will, in turn, also mate after one month, producing another pair, and so on. Rabbits never die. How many pairs of rabbits exist after one year?

- Every point denotes one rabbit pair.
- A grey point denotes a newborn pair (and not ready to reproduce).
- A red point denotes a mature, reproducing pair.

5 of 39

Suppose we have two newly-born rabbits, one female and one male. Suppose these rabbits produce another pair of female and male rabbits after one month. These newly-born rabbits will, in turn, also mate after one month, producing another pair, and so on. Rabbits never die. How many pairs of rabbits exist after one year?

- Every point denotes one rabbit pair.
- A grey point denotes a newborn pair (and not ready to reproduce).
- A red point denotes a mature, reproducing pair.

5 of 39

A Quick Look at Fibonacci Sequence

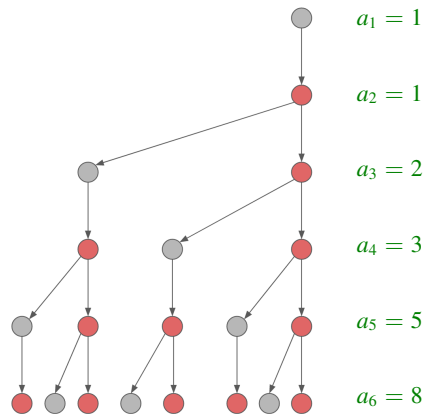
During 13th century, in Liber Abaci, Fibonacci* poses the following question (paraphrasing):

Suppose we have two newly-born rabbits, one female and one male. Suppose these rabbits produce another pair of female and male rabbits after one month. These newly-born rabbits will, in turn, also mate after one month, producing another pair; and so on. Rabbits never die. How many pairs of rabbits exist after one year?

The figure to the right illustrates this process.

- Every point denotes one rabbit pair.
- A grey point denotes a newborn pair (and not ready to reproduce).
- A red point denotes a mature, reproducing pair.

Fibonacci's Rabbits



*discovered earlier by Indian scholars (Gopāla, before 1135), studying rhythmic patterns

Closed vs Recursive Formula for Sequences

We often need to specify a rule for the general term in the sequence — we have two options:

Definition 2 (Closed Formula and Recursive Definition)

- A **closed formula** for a sequence a_n is a formula for a_n using a fixed, finite number of operations on n).
- A **recursive definition** for a sequence (a_n) consists of a **recurrence relation**: an equation relating the current term in the sequence, (a_n) , to earlier terms in the sequence, (a_{n-1}) , (a_{n-2}) , ... (i.e., terms with smaller index) and **initial/terminal condition(s)**.

Example

The Fibonacci sequence $(a_n) = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots)$ has closed formula

$$a_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

and recursive formula

$$\underbrace{a_n = a_{n-1} + a_{n-2}}_{\text{recurrence relation}} \quad \text{and} \quad \underbrace{a_0 = 0, \quad a_1 = 1}_{\text{terminal conditions}}$$

Closed vs Recursive Formula for Sequences

We often need to specify a rule for the general term in the sequence — we have two options:

Definition 2 (Closed Formula and Recursive Definition)

- A **closed formula** for a sequence a_n is a formula for a_n using a fixed, finite number of operations on n).
- A **recursive definition** for a sequence (a_n) consists of a **recurrence relation**: an equation relating the current term in the sequence, (a_n) , to earlier terms in the sequence, (a_{n-1}) , (a_{n-2}) , ... (i.e., terms with smaller index) and **initial/terminal condition(s)**.

Example

The Fibonacci sequence $(a_n) = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots)$ has closed formula

$$a_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

and recursive formula

$$\underbrace{a_n = a_{n-1} + a_{n-2}}_{\text{recurrence relation}} \quad \text{and} \quad \underbrace{a_0 = 0, \quad a_1 = 1}_{\text{terminal conditions}}$$

Closed vs Recursive Formula for Sequences

We often need to specify a rule for the general term in the sequence — we have two options:

Definition 2 (Closed Formula and Recursive Definition)

- A **closed formula** for a sequence a_n is a formula for a_n using a fixed, finite number of operations on n).
- A **recursive definition** for a sequence (a_n) consists of a **recurrence relation**: an equation relating the current term in the sequence, (a_n) , to earlier terms in the sequence, (a_{n-1}) , (a_{n-2}) , ... (i.e., terms with smaller index) and **initial/terminal condition(s)**.

Example

The Fibonacci sequence $(a_n) = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots)$ has closed formula

$$a_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

Hard to obtain, easy to use

and recursive formula

$$\underbrace{a_n = a_{n-1} + a_{n-2}}_{\text{recurrence relation}}$$

and

$$\underbrace{a_0 = 0, \quad a_1 = 1}_{\text{terminal conditions}}$$

Easy to obtain,
hard to use

Computing Fibonacci Sequence using Closed Formula

Compute the first 7 terms of the Fibonacci sequence using the closed formula

$$a_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

```
1 import math
2
3 for n in range(7):
4
5     tmp_1 = (1 + math.sqrt(5)) / 2
6     tmp_2 = (1 - math.sqrt(5)) / 2
7
8     a_n = (tmp_1**n - tmp_2**n) / math.sqrt(5)
9
10 print(n, round(a_n))
```



Computing Fibonacci Sequence using Closed Formula

Compute the first 7 terms of the Fibonacci sequence using the closed formula

$$a_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

```

2 ●
1 import math
2
3 for n in range(7):
4     [ ]
5     tmp_1 = (1 + math.sqrt(5)) / 2
6     tmp_2 = (1 - math.sqrt(5)) / 2
7     [ ]
8     a_n = (tmp_1**n - tmp_2**n) / math.sqrt(5)
9     [ ]
10    print(n, round(a_n))

```

0	0
1	1
2	1
3	2
4	3
5	5
6	8



Computing Fibonacci Sequence using Recursive Formula

Compute the first 7 terms of the Fibonacci sequence using the recursive formula

```
3 1 previous_previous = 0
2 previous = 1
3
4 for n in range(7):
5
6     if n == 0:          # terminal condition n=0
7         current = 0
8     elif n == 1:       # terminal condition n=1
9         current = 1
10    else:               # recursive formula n>1
11        current = previous + previous_previous
12
13        # leapfrog values
14        previous_previous = previous
15        previous = current
16
17    print(n, current)
```



Computing Fibonacci Sequence using Recursive Formula

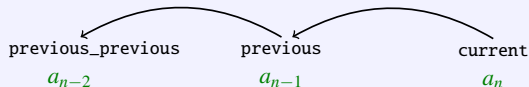
Compute the first 7 terms of the Fibonacci sequence using the recursive formula

```

1 previous_previous = 0
2 previous = 1
3
4 for n in range(7):
5
6     if n == 0:           # terminal condition n=0
7         current = 0
8     elif n == 1:        # terminal condition n=1
9         current = 1
10    else:                # recursive formula n>1
11        current = previous + previous_previous
12
13    # leapfrog values
14    previous_previous = previous
15    previous = current
16
17 print(n, current)

```

0	0
1	1
2	1
3	2
4	3
5	5
6	8



Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

$$a_0 = 3 \quad \text{(given terminal condition)}$$

$$a_1 = 4 \quad \text{(given terminal condition)}$$

Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

$$a_0 = 3 \quad (\text{given terminal condition})$$

$$a_1 = 4 \quad (\text{given terminal condition})$$

$$a_2 = 2 \cdot 4 - 3 = 5 \quad (\text{use } n = 2 \text{ in recursive formula})$$

Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

$$a_0 = 3 \quad (\text{given terminal condition})$$

$$a_1 = 4 \quad (\text{given terminal condition})$$

$$a_2 = 2 \cdot 4 - 3 = 5 \quad (\text{use } n = 2 \text{ in recursive formula})$$

$$a_3 = 2 \cdot 5 - 4 = 6 \quad (\text{use } n = 3 \text{ in recursive formula})$$

Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

$$a_0 = 3 \quad (\text{given terminal condition})$$

$$a_1 = 4 \quad (\text{given terminal condition})$$

$$a_2 = 2 \cdot 4 - 3 = 5 \quad (\text{use } n = 2 \text{ in recursive formula})$$

$$a_3 = 2 \cdot 5 - 4 = 6 \quad (\text{use } n = 3 \text{ in recursive formula})$$

$$a_4 = 2 \cdot 6 - 5 = 7 \quad (\text{use } n = 4 \text{ in recursive formula})$$

Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

$$a_0 = 3 \quad (\text{given terminal condition})$$

$$a_1 = 4 \quad (\text{given terminal condition})$$

$$a_2 = 2 \cdot 4 - 3 = 5 \quad (\text{use } n = 2 \text{ in recursive formula})$$

$$a_3 = 2 \cdot 5 - 4 = 6 \quad (\text{use } n = 3 \text{ in recursive formula})$$

$$a_4 = 2 \cdot 6 - 5 = 7 \quad (\text{use } n = 4 \text{ in recursive formula})$$

$$a_5 = 2 \cdot 7 - 6 = 8 \quad (\text{use } n = 5 \text{ in recursive formula})$$

Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

$a_0 = 3$	(given terminal condition)
$a_1 = 4$	(given terminal condition)
$a_2 = 2 \cdot 4 - 3 = 5$	(use $n = 2$ in recursive formula)
$a_3 = 2 \cdot 5 - 4 = 6$	(use $n = 3$ in recursive formula)
$a_4 = 2 \cdot 6 - 5 = 7$	(use $n = 4$ in recursive formula)
$a_5 = 2 \cdot 7 - 6 = 8$	(use $n = 5$ in recursive formula)
$a_6 = 2 \cdot 8 - 7 = 9$	(use $n = 6$ in recursive formula)

Example

Example 3

Find a_6 in the sequence defined by $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 3$ and $a_1 = 4$.

Solution. Using $n = 6$, we know that $a_6 = 2a_5 - a_4$. So to find a_6 we need to find a_5 and a_4 . And we repeat this process down to a_0 and a_1 . We will use the approach when we define functions.

But for now, we will determine a_6 by starting at a_0 and a_1 , and working upwards towards a_6 .

$a_0 = 3$	(given terminal condition)
$a_1 = 4$	(given terminal condition)
$a_2 = 2 \cdot 4 - 3 = 5$	(use $n = 2$ in recursive formula)
$a_3 = 2 \cdot 5 - 4 = 6$	(use $n = 3$ in recursive formula)
$a_4 = 2 \cdot 6 - 5 = 7$	(use $n = 4$ in recursive formula)
$a_5 = 2 \cdot 7 - 6 = 8$	(use $n = 5$ in recursive formula)
$a_6 = 2 \cdot 8 - 7 = 9$	(use $n = 6$ in recursive formula)

Note that in this case a closed formula for a_n exists. Namely, $a_n = n + 3$.

A closed formula is easier to use to calculate a general term, but it is often much harder, if not impossible, to derive.

Computing Sequence using Closed Formula

First 7 terms of the sequence using the closed formula

$$a_n = n + 3$$

```
5 ●  
1 for n in range(7):  
2  
3     a_n = n + 3  
4  
5     print(n, a_n)
```



Computing Sequence using Closed Formula

First 7 terms of the sequence using the closed formula

$$a_n = n + 3$$

```
6 ●  
1 for n in range(7):  
2       
3     a_n = n + 3  
4       
5     print(n, a_n)
```

```
0 3  
1 4  
2 5  
3 6  
4 7  
5 8  
6 9
```



Computing Sequence using Recursive Formula

Compute the first 7 terms of the Fibonacci sequence using the recursive formula

```
7 1 previous_previous = 3
2 previous = 4
3
4 for n in range(7):
5
6     if n == 0:          # terminal condition n=0
7         current = 3
8     elif n == 1:        # terminal condition n=1
9         current = 4
10    else:                # recursive formula n>1
11        current = 2 * previous - previous_previous
12
13        # leapfrog values
14        previous_previous = previous
15        previous = current
16
17    print(n, current)
```



Computing Sequence using Recursive Formula

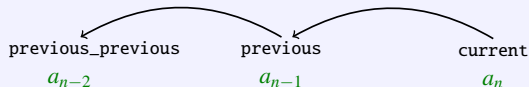
Compute the first 7 terms of the Fibonacci sequence using the recursive formula

```

1 previous_previous = 3
2 previous = 4
3
4 for n in range(7):
5
6     if n == 0:           # terminal condition n=0
7         current = 3
8     elif n == 1:        # terminal condition n=1
9         current = 4
10    else:                # recursive formula n>1
11        current = 2 * previous - previous_previous
12
13    # leapfrog values
14    previous_previous = previous
15    previous = current
16
17 print(n, current)

```

0	3
1	4
2	5
3	6
4	7
5	8
6	9



Summation Notation

- The $\sum$ operator is used to denote the addition of elements from a sequence/list.
- It can be implemented using a **for** loop in Python/Java/Processing.

Example 4

$$\underbrace{\sum_{k=1}^{10} \left[k^2 \right]}_{\text{Determine the value of expression within the brackets as } k = 1, 2, 3, \dots, 10 \text{ and add all the results.}} = \underbrace{1^2}_{k=1} + \underbrace{2^2}_{k=2} + \underbrace{3^2}_{k=3} + \underbrace{4^2}_{k=4} + \dots + \underbrace{10^2}_{k=10}$$

“Determine the value of expression within the brackets as $k = 1, 2, 3, \dots, 10$ and add all the results.”

$$= 1 + 4 + 9 + 16 + 25 + 36 + \dots + 100 = 385$$

```

9 •
1 result = 0                # start result with zero - why?
2 for k in range(1,11):
3     term = k*k
4     result += term        # shorthand for result = result + term
5
6 print(result)

```

385



Product Notation

- The $\prod$ operator is used to denote the product of elements from a sequence/list.
- It can be implemented using a **for** loop in Python/Java/Processing.

Example 5

$$\prod_{k=1}^{10} [k^2] = \underbrace{1^2}_{k=1} \times \underbrace{2^2}_{k=2} \times \underbrace{3^2}_{k=3} \times \underbrace{4^2}_{k=4} \times \cdots \times \underbrace{10^2}_{k=10}$$

“Determine the value of expression within the brackets as $k = 1, 2, 3, \dots, 10$ and multiply all the results.”

$$= 1 \times 4 \times 9 \times 16 \times 25 \times 36 \times \cdots \times 100 = 13,168,189,440,000$$

```

10 ●
1 result = 1                # start result with one - why?
2 for k in range(1,11):
3     term = k*k
4     result *= term        # shorthand for result = result * term
5
6 print(result)

```

13168189440000



Review Exercises 1 (Sequences)

Sequences

Question 1:

Expand the following sums

(a) $\sum_{k=4}^7 k$

(b) $\sum_{k=1}^5 (k^1 - 1)$

(c) $\sum_{n=1}^4 (10^n)$

(d) $\sum_{k=1}^5 (k^1 - 1)$

Question 2:

Write the following expressions using summation notation

(a) $2 + 4 + 6 + 8 + 10$

(b) $1 + 4 + 7 + 10$

(c) $\frac{1}{4} + \frac{1}{2} + 1 + 2 + 4$

Question 3:

Expand the following sums

(a) $\prod_{k=-4}^4 k$

(b) $\prod_{k=1}^4 (k^1 - 1)$

(c) $\prod_{k \in S} (-1)^k$ where $S = \{2, 4, 6, 7\}$.

Question 4:

For each of the following sequences, determine a recursive definition.

(a) $2, 4, 6, 10, 16, 26, 42, \dots$

ⓑ $5, 6, 11, 17, 28, 45, 73, \dots$

ⓒ $0, 0, 0, 0, 0, 0, 0, \dots$

Question 5:

Show that $a_n = 3 \cdot 2^n + 7 \cdot 5^n$ is a solution to the recurrence relation $a_n = 7a_{n-1} - 10a_{n-2}$. What would the initial conditions need to be for this to be the closed formula for the sequence?

Outline

1. Sequences	2
2. Arithmetic and Geometric Progressions	16
2.1. Definition of Arithmetic and Geometric Progression	17
2.2. Partial Sums of AP and GP	19
3. Implementing Sequence Collections in Python	25
3.1. Common Concepts	26
3.2. Lists	27
3.3. Tuples	37
4. Strings	38

Arithmetic Progression/Sequence

Definition 6 (Arithmetic Progression/Sequence (AP))

A sequence is called **arithmetic** if the terms of the sequence differ by a constant.

Suppose the initial term (a_0) of the sequence is a and the **common difference** is d , then we have sequence

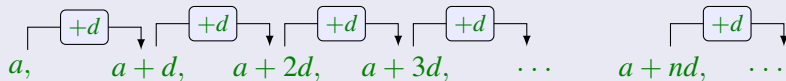
$$a, \quad a + d, \quad a + 2d, \quad a + 3d, \quad \dots \quad a + nd, \quad \dots$$

Arithmetic Progression/Sequence

Definition 6 (Arithmetic Progression/Sequence (AP))

A sequence is called **arithmetic** if the terms of the sequence differ by a constant.

Suppose the initial term (a_0) of the sequence is a and the **common difference** is d , then we have sequence

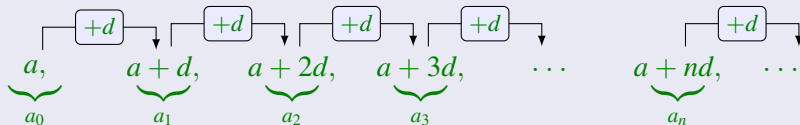


Arithmetic Progression/Sequence

Definition 6 (Arithmetic Progression/Sequence (AP))

A sequence is called **arithmetic** if the terms of the sequence differ by a constant.

Suppose the initial term (a_0) of the sequence is a and the **common difference** is d , then we have sequence

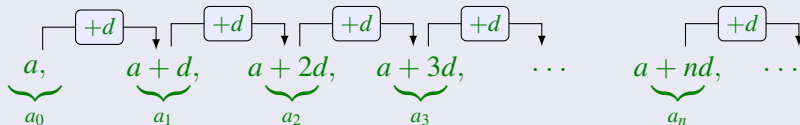


Arithmetic Progression/Sequence

Definition 6 (Arithmetic Progression/Sequence (AP))

A sequence is called **arithmetic** if the terms of the sequence differ by a constant.

Suppose the initial term (a_0) of the sequence is a and the **common difference** is d , then we have sequence



Recursive definition: $a_n = a_{n-1} + d$ with $a_0 = a$.

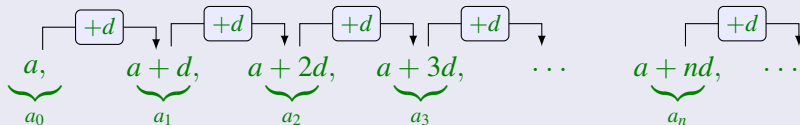
Closed formula: $a_n = a + dn$.

Arithmetic Progression/Sequence

Definition 6 (Arithmetic Progression/Sequence (AP))

A sequence is called **arithmetic** if the terms of the sequence differ by a constant.

Suppose the initial term (a_0) of the sequence is a and the **common difference** is d , then we have sequence



Recursive definition: $a_n = a_{n-1} + d$ with $a_0 = a$.

Closed formula: $a_n = a + dn$.

Example 7

Find recursive definitions and closed formulas for the sequences below. Assume the first term listed is a_0 .

- $2, 5, 8, 11, 14, \dots$
- $50, 43, 36, 29, \dots$

Geometric Progression/Sequence

Definition 8 (Geometric Progression/Sequence (GP))

A sequence is called **geometric** if the ratio between successive terms is constant.

Suppose the initial term a_0 is a and the **common ratio** is r . Then we have, sequence

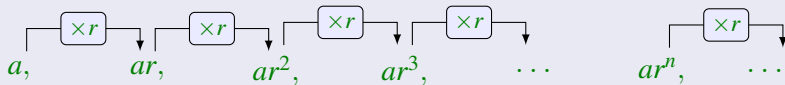
$$a, \quad ar, \quad ar^2, \quad ar^3, \quad \dots \quad ar^n, \quad \dots$$

Geometric Progression/Sequence

Definition 8 (Geometric Progression/Sequence (GP))

A sequence is called **geometric** if the ratio between successive terms is constant.

Suppose the initial term a_0 is a and the **common ratio** is r . Then we have, sequence

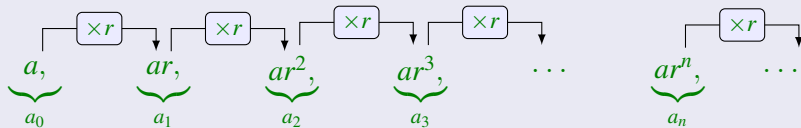


Geometric Progression/Sequence

Definition 8 (Geometric Progression/Sequence (GP))

A sequence is called **geometric** if the ratio between successive terms is constant.

Suppose the initial term a_0 is a and the **common ratio** is r . Then we have, sequence

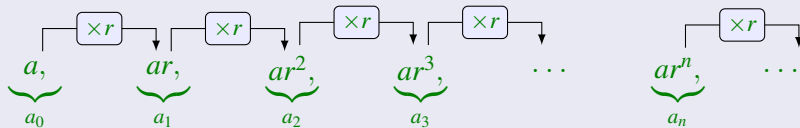


Geometric Progression/Sequence

Definition 8 (Geometric Progression/Sequence (GP))

A sequence is called **geometric** if the ratio between successive terms is constant.

Suppose the initial term a_0 is a and the **common ratio** is r . Then we have, sequence



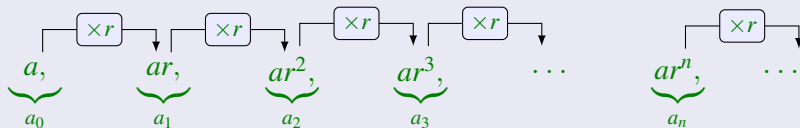
Recursive definition: $a_n = ra_{n-1}$ with $a_0 = a$.

Closed formula: $a_n = ar^n$.

Geometric Progression/Sequence

Definition 8 (Geometric Progression/Sequence (GP))

A sequence is called **geometric** if the ratio between successive terms is constant. Suppose the initial term a_0 is a and the **common ratio** is r . Then we have, sequence



Recursive definition: $a_n = ra_{n-1}$ with $a_0 = a$.

Closed formula: $a_n = ar^n$.

Example 9

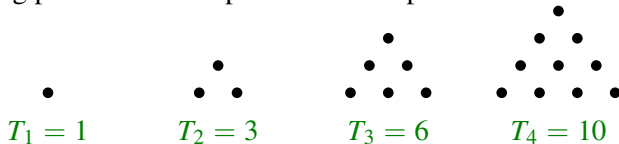
Find the recursive and closed formula for the sequences below. Again, the first term listed is a_0 .

- $3, 6, 12, 24, 48, \dots$

- $27, 9, 3, 1, 1/3, \dots$

Motivation

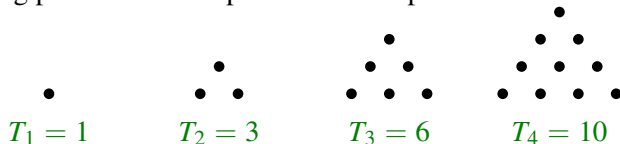
Look at the sequence $(T_n)_{n \geq 1}$ which starts $1, 3, 6, 10, 15, \dots$. These are called the **triangular numbers** since they represent the number of dots in an equilateral triangle (think of how you arrange 10 bowling pins: a row of 4 plus a row of 3 plus a row of 2 and a row of 1).



- Is this sequence arithmetic?
- Is the sequence geometric?

Motivation

Look at the sequence $(T_n)_{n \geq 1}$ which starts $1, 3, 6, 10, 15, \dots$. These are called the **triangular numbers** since they represent the number of dots in an equilateral triangle (think of how you arrange 10 bowling pins: a row of 4 plus a row of 3 plus a row of 2 and a row of 1).



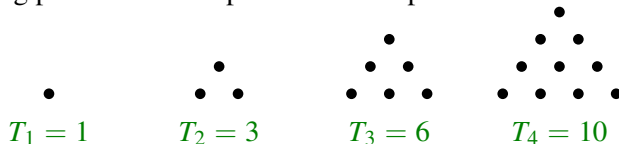
- Is this sequence arithmetic?

No, since $3 - 1 = 2$ and $6 - 3 = 3 \neq 2$, so there is no common difference.

- Is the sequence geometric?

Motivation

Look at the sequence $(T_n)_{n \geq 1}$ which starts $1, 3, 6, 10, 15, \dots$. These are called the **triangular numbers** since they represent the number of dots in an equilateral triangle (think of how you arrange 10 bowling pins: a row of 4 plus a row of 3 plus a row of 2 and a row of 1).



- Is this sequence arithmetic?

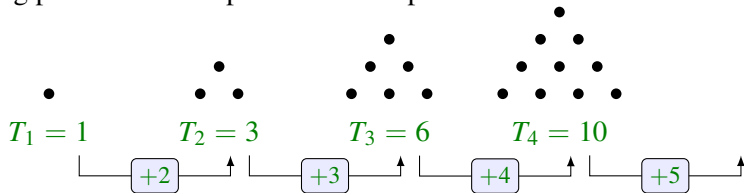
No, since $3 - 1 = 2$ and $6 - 3 = 3 \neq 2$, so there is no common difference.

- Is the sequence geometric?

No. $3/1 = 3$ but $6/3 = 2$, so there is no common ratio.

Motivation

Look at the sequence $(T_n)_{n \geq 1}$ which starts $1, 3, 6, 10, 15, \dots$. These are called the **triangular numbers** since they represent the number of dots in an equilateral triangle (think of how you arrange 10 bowling pins: a row of 4 plus a row of 3 plus a row of 2 and a row of 1).



- Is this sequence arithmetic?

No, since $3 - 1 = 2$ and $6 - 3 = 3 \neq 2$, so there is no common difference.

- Is the sequence geometric?

No. $3/1 = 3$ but $6/3 = 2$, so there is no common ratio.

- Notice that the differences between terms generate an arithmetic sequence: $2, 3, 4, 5, 6, \dots$

This says that the n th term of the triangular sequence is the sum of the first n terms in the sequence $1, 2, 3, 4, 5, \dots$, i.e, the triangular sequence is a **sequence of partial sums**.

Summing Arithmetic Sequences: Reverse and Add

I

Example 10

Find the sum: $2 + 5 + 8 + 11 + 14 + \cdots + 470$.

Solution. If we add the first and last terms, we get 472. The second term and second-to-last term also add up to 472. To keep track of everything, we might express this as follows. Call the sum S . Then,

$$\begin{array}{r}
 S = 2 + 5 + 8 + \cdots + 467 + 470 \\
 + S = 470 + 467 + 464 + \cdots + 5 + 2 \\
 \hline
 2S = 472 + 472 + 472 + \cdots + 472 + 472
 \end{array}$$

Hence, to find $2S$ then we add 472 to itself a number of times. What number?

We need to decide how many terms are in the sum. Since the terms form an arithmetic sequence, the n th term in the sum (counting 2 as the 0th term) can be expressed as $2 + 3n$. If $2 + 3n = 470$ then $n = 156$. So n ranges from 0 to 156, giving 157 terms in the sum. This is the number of 472's in the sum for $2S$. Thus

$$2S = 157 \times 472 = 74104 \quad \implies \quad S = \frac{74104}{2} = 37052$$

Summing Arithmetic Sequences: Reverse and Add

I

Example 10

Find the sum: $2 + 5 + 8 + 11 + 14 + \cdots + 470$.

Solution. If we add the first and last terms, we get 472. The second term and second-to-last term also add up to 472. To keep track of everything, we might express this as follows. Call the sum S . Then,

$$\begin{array}{r}
 S = 2 + 5 + 8 + \cdots + 467 + 470 \\
 + S = 470 + 467 + 464 + \cdots + 5 + 2 \\
 \hline
 2S = 472 + 472 + 472 + \cdots + 472 + 472
 \end{array}$$

Hence, to find $2S$ then we add 472 to itself a number of times. What number?

We need to decide how many terms are in the sum. Since the terms form an arithmetic sequence, the n th term in the sum (counting 2 as the 0th term) can be expressed as $2 + 3n$. If $2 + 3n = 470$ then $n = 156$. So n ranges from 0 to 156, giving 157 terms in the sum. This is the number of 472's in the sum for $2S$. Thus

$$2S = 157 \times 472 = 74104 \quad \implies \quad S = \frac{74104}{2} = 37052$$

Summing Arithmetic Sequences: Reverse and Add

I

Example 10

Find the sum: $2 + 5 + 8 + 11 + 14 + \cdots + 470$.

Solution. If we add the first and last terms, we get 472. The second term and second-to-last term also add up to 472. To keep track of everything, we might express this as follows. Call the sum S . Then,

$$\begin{array}{r}
 S = 2 + 5 + 8 + \cdots + 467 + 470 \\
 + S = 470 + 467 + 464 + \cdots + 5 + 2 \\
 \hline
 2S = 472 + 472 + 472 + \cdots + 472 + 472
 \end{array}$$

Hence, to find $2S$ then we add 472 to itself a number of times. What number?

We need to decide how many terms are in the sum. Since the terms form an arithmetic sequence, the n th term in the sum (counting 2 as the 0th term) can be expressed as $2 + 3n$. If $2 + 3n = 470$ then $n = 156$. So n ranges from 0 to 156, giving 157 terms in the sum. This is the number of 472's in the sum for $2S$. Thus

$$2S = 157 \times 472 = 74104 \quad \implies \quad S = \frac{74104}{2} = 37052$$

Summing Arithmetic Sequences: Reverse and Add

I

Example 10

Find the sum: $2 + 5 + 8 + 11 + 14 + \cdots + 470$.

Solution. If we add the first and last terms, we get 472. The second term and second-to-last term also add up to 472. To keep track of everything, we might express this as follows. Call the sum S . Then,

$$\begin{array}{r}
 S = 2 + 5 + 8 + \cdots + 467 + 470 \\
 + S = 470 + 467 + 464 + \cdots + 5 + 2 \\
 \hline
 2S = 472 + 472 + 472 + \cdots + 472 + 472
 \end{array}$$

Hence, to find $2S$ then we add 472 to itself a number of times. What number?

We need to decide how many terms are in the sum. Since the terms form an arithmetic sequence, the n th term in the sum (counting 2 as the 0th term) can be expressed as $2 + 3n$. If $2 + 3n = 470$ then $n = 156$. So n ranges from 0 to 156, giving 157 terms in the sum. This is the number of 472's in the sum for $2S$. Thus

$$2S = 157 \times 472 = 74104 \quad \implies \quad S = \frac{74104}{2} = 37052$$

Summing Arithmetic Sequences: Reverse and Add

The process covered in the previous slide will work for any sum of arithmetic sequences.

- STEP 1 Call the sum S .
- STEP 2 Reverse and add.
- STEP 3 This produces a single number added to itself many times.
- STEP 4 Determine the number of times.
- STEP 5 Multiply. Divide by 2. Done

Definition 11 (Arithmetic Series)

The sum of the terms of the arithmetic sequence

$$S_n = [a] + [a + d] + [a + 2d] + \cdots + [a + nd]$$

is called an **arithmetic series** and is given by

$$S_n = (n + 1)a + \frac{dn(n + 1)}{2}$$

Summing Arithmetic Sequences: Reverse and Add

The process covered in the previous slide will work for any sum of arithmetic sequences.

- STEP 1 Call the sum S .
- STEP 2 Reverse and add.
- STEP 3 This produces a single number added to itself many times.
- STEP 4 Determine the number of times.
- STEP 5 Multiply. Divide by 2. Done

Definition 11 (Arithmetic Series)

The sum of the terms of the arithmetic sequence

$$S_n = [a] + [a + d] + [a + 2d] + \cdots + [a + nd]$$

is called an **arithmetic series** and is given by

$$S_n = (n + 1)a + \frac{dn(n + 1)}{2}$$

Summing Geometric Sequences: Multiply and Subtract

I

To find the sum of a geometric sequence, we cannot just reverse and add. Instead we multiply and subtract:

Example 12

What is $3 + 6 + 12 + 24 + \cdots + 12288$?

STEP 1 Call the sum S .

STEP 2 Multiply each term by the common ratio, $r = 2$

STEP 3 Subtract, and solve for S .

Summing Geometric Sequences: Multiply and Subtract

I

To find the sum of a geometric sequence, we cannot just reverse and add. Instead we multiply and subtract:

Example 12

What is $3 + 6 + 12 + 24 + \cdots + 12288$?

This terms in the sum are from a geometric progression with initial term, $a_0 = 3$, and common ratio, $r = 2$.

STEP 1 Call the sum S .

STEP 2 Multiply each term by the common ratio, $r = 2$

STEP 3 Subtract, and solve for S .

Summing Geometric Sequences: Multiply and Subtract

I

To find the sum of a geometric sequence, we cannot just reverse and add. Instead we multiply and subtract:

Example 12

What is $3 + 6 + 12 + 24 + \cdots + 12288$?

This terms in the sum are from a geometric progression with initial term, $a_0 = 3$, and common ratio, $r = 2$.

STEP 1 Call the sum S .

STEP 2 Multiply each term by the common ratio, $r = 2$

STEP 3 Subtract, and solve for S .

$$S = 3 + 6 + 12 + 24 + \cdots + 12288$$

Summing Geometric Sequences: Multiply and Subtract

I

To find the sum of a geometric sequence, we cannot just reverse and add. Instead we multiply and subtract:

Example 12

What is $3 + 6 + 12 + 24 + \cdots + 12288$?

This terms in the sum are from a geometric progression with initial term, $a_0 = 3$, and common ratio, $r = 2$.

STEP 1 Call the sum S .

STEP 2 Multiply each term by the common ratio, $r = 2$

STEP 3 Subtract, and solve for S .

$$\begin{array}{rcl}
 S & = & 3 + 6 + 12 + 24 + \cdots + 12288 \\
 2S & = & 6 + 12 + 24 + \cdots + 12288 \quad + 24576
 \end{array}$$

Summing Geometric Sequences: Multiply and Subtract

I

To find the sum of a geometric sequence, we cannot just reverse and add. Instead we multiply and subtract:

Example 12

What is $3 + 6 + 12 + 24 + \cdots + 12288$?

This terms in the sum are from a geometric progression with initial term, $a_0 = 3$, and common ratio, $r = 2$.

STEP 1 Call the sum S .

STEP 2 Multiply each term by the common ratio, $r = 2$

STEP 3 Subtract, and solve for S .

$$\begin{array}{rcl}
 S & = & 3 + 6 + 12 + 24 + \cdots + 12288 \\
 2S & = & 6 + 12 + 24 + \cdots + 12288 \quad +24576 \\
 \hline
 -S & = & 3 + 0 + 0 + 0 + \cdots + 0 \quad -24576 \\
 \hline
 -S & = & 3 - 24576 \quad \implies \quad S = 24573
 \end{array}$$

Summing Geometric Sequences: Multiply and Subtract

II

Definition 13 (Geometric Series)

The sum of the terms of the geometric sequence

$$S_n = [a] + [ar] + [ar^2] + \cdots + [ar^n]$$

is called a **geometric series** and is given by

$$S_n = \frac{a(1 - r^{n+1})}{1 - r}$$

- In the special case of $-1 < r < 1$ the terms in the geometric sequence tends towards zero fast enough that the sum of the series tends to the finite value

$$S_\infty = \lim_{n \rightarrow \infty} S_n = \lim_{n \rightarrow \infty} \frac{a}{1 - r}$$

since $r^{n+1} \rightarrow 0$ as $n \rightarrow \infty$.

Summing Geometric Sequences: Multiply and Subtract

II

Definition 13 (Geometric Series)

The sum of the terms of the geometric sequence

$$S_n = [a] + [ar] + [ar^2] + \cdots + [ar^n]$$

is called a **geometric series** and is given by

$$S_n = \frac{a(1 - r^{n+1})}{1 - r}$$

- In the special case of $-1 < r < 1$ the terms in the geometric sequence tends towards zero fast enough that the sum of the series tends to the finite value

$$S_\infty = \lim_{n \rightarrow \infty} S_n = \lim_{n \rightarrow \infty} \frac{a}{1 - r}$$

since $r^{n+1} \rightarrow 0$ as $n \rightarrow \infty$.

Question 1:

Consider the sequence $5, 9, 13, 17, 21, \dots$ with $a_1 = 5$

- (a) Give a recursive definition for the sequence.
- (b) Give a closed formula for the n th term of the sequence.
- (c) Is 2013 a term in the sequence? Explain.
- (d) How many terms does the sequence $5, 9, 13, 17, 21, \dots, 533$ have?
- (e) Determine the sum: $5 + 9 + 13 + 17 + 21 + \dots + 533$. Show your work.
- (f) Use what you found above to find b_n , the n^{th} term of $1, 6, 15, 28, 45, \dots$, where $b_0 = 1$

Outline

1. Sequences	2
2. Arithmetic and Geometric Progressions	16
2.1. Definition of Arithmetic and Geometric Progression	17
2.2. Partial Sums of AP and GP	19
3. Implementing Sequence Collections in Python	25
3.1. Common Concepts	26
3.2. Lists	27
3.3. Tuples	37
4. Strings	38

Math vs. Programming (Python/Processing/Java/...)

Computers are finite

In mathematics we can define a sequence, just like

$$a_n = 2^n, \text{ for } n \geq 0 \qquad 0, 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, \dots$$

and have no concerns that it has infinite length or that the values become arbitrary large. This is not the case when programming — we (or the language designers) need to deal with both of these issues.

Infinite length sequences in Mathematics → (usually) Finite length sequences in Python

Programmers need standard tasks/operations

- **Indexing** — Each position in a sequence is given a unique position/index, so we can access/change a single element by referring to its index.
- **Slicing** — Given a sequence collection we want to create a copy of part of that sequence.
- **Iterating over** — Looping over all elements (**for** loops and **list comprehensions**).
- **Filtering** — Given a collection construct a new collection containing only elements that satisfy a condition.

Python Implementation — Sets vs Lists

```
11 ●
1 S = set()      # cannot use {}
2 L = []         # here we can use list() or []
3 print(S, L)
4
5 S.add(3)       # we ADD to a set
6 L.append(3)    # but we APPEND to END of list
7 print(S, L)
8
9 S.add(3)       # elements are distinct
10 L.append(3)
11 print(S, L)
12
13 S.add("Hello") # can store mixture of data types
14 L.append("Hello") # can store mixture of data types
15 print(S, L)
16
17 S.add("All")   # unordered
18 L.append("All") # ordered
19 print(S, L)
```



Python Implementation — Sets vs Lists

```
12 ●
1 ● S = set()    # cannot use {}
2 ● L = []       # here we can use list() or []
3 ● print(S, L)
4
5 S.add(3)       # we ADD to a set
6 L.append(3)    # but we APPEND to END of list
7 print(S, L)
8
9 S.add(3)       # elements are distinct
10 L.append(3)
11 print(S, L)
12
13 S.add("Hello") # can store mixture of data types
14 L.append("Hello") # can store mixture of data types
15 print(S, L)
16
17 S.add("All")   # unordered
18 L.append("All") # ordered
19 print(S, L)
```

set() []



Python Implementation — Sets vs Lists

```
13 ● S = set()    # cannot use {}
1  L = []        # here we can use list() or []
2
3  print(S, L)
4
5  ● S.add(3)      # we ADD to a set
6  ● L.append(3)   # but we APPEND to END of list
7  ● print(S, L)
8
9  S.add(3)       # elements are distinct
10 L.append(3)
11 print(S, L)
12
13 S.add("Hello")  # can store mixture of data types
14 L.append("Hello") # can store mixture of data types
15 print(S, L)
16
17 S.add("All")    # unordered
18 L.append("All") # ordered
19 print(S, L)
```

```
set() []
{3} [3]
```



Python Implementation — Sets vs Lists

```

14 ● S = set()    # cannot use {}
1  L = []        # here we can use list() or []
2  print(S, L)
3
4
5  S.add(3)       # we ADD to a set
6  L.append(3)    # but we APPEND to END of list
7  print(S, L)
8
9  ● S.add(3)     # elements are distinct
10 ● L.append(3)
11 ● print(S, L)
12
13 S.add("Hello") # can store mixture of data types
14 L.append("Hello") # can store mixture of data
15 print(S, L)
16
17 S.add("All")   # unordered
18 L.append("All") # ordered
19 print(S, L)

```



```

set() []
{3} [3]
{3} [3, 3]

```

Python Implementation — Sets vs Lists

```

15 ● S = set()    # cannot use {}
1  L = []        # here we can use list() or []
2  print(S, L)
3
4
5  S.add(3)       # we ADD to a set
6  L.append(3)    # but we APPEND to END of list
7  print(S, L)
8
9  S.add(3)       # elements are distinct
10 L.append(3)
11 print(S, L)
12
13 ● S.add("Hello") # can store mixture of data
14 ● L.append("Hello") # can store mixture of data
15 ● print(S, L)
16
17 S.add("All")    # unordered
18 L.append("All") # ordered
19 print(S, L)

```



```

set() []
{3} [3]
{3} [3, 3]
{'Hello', 3} [3, 3, 'Hello']

```

Python Implementation — Sets vs Lists

```

16 ● S = set()    # cannot use {}
1  L = []        # here we can use list() or []
2
3  print(S, L)
4
5  S.add(3)      # we ADD to a set
6  L.append(3)   # but we APPEND to END of list
7  print(S, L)
8
9  S.add(3)      # elements are distinct
10 L.append(3)
11 print(S, L)
12
13 S.add("Hello") # can store mixture of data
14 L.append("Hello") # can store mixture of data
15 print(S, L)
16
17 ● S.add("All") # unordered
18 ● L.append("All") # ordered
19 ● print(S, L)

```



```

set() []
{3} [3]
{3} [3, 3]
{'Hello', 3} [3, 3, 'Hello']
{'Hello', 'All', 3} [3, 3, 'Hello', 'All']

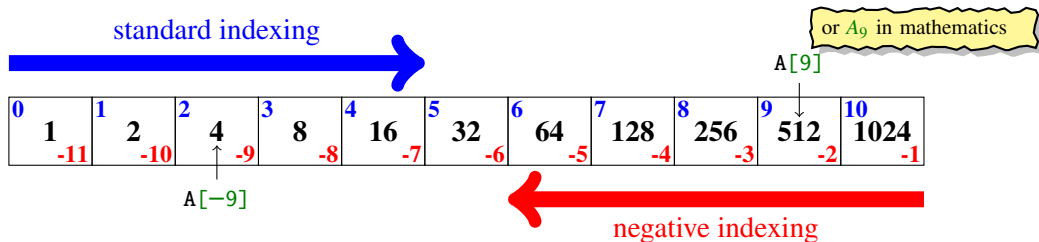
```

Indexing

To help illustrate indexing we will define a list containing the powers of 2 up to and including 2^{10} .

```
17 ● A = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]
```

- The collection (a **list**) is **ordered** — so we can talk about which data value (item/element) comes before/after another data value.
- In addition, each data value has a position, called **index**, which counts from the left of the list. Python is zero-based language so index starts at zero.
- Python, also support **negative indexing** which counts backwards from the end of the list.



Slicing

I

Definition 14 (Slicing)

Slicing is a compact syntax to construct a sub-sequence collection from a larger collection. A slice consists of

`[start:end:step]`

where

- `start` — the starting index (inclusive). Defaults to 0 (i.e., start of the collection) if omitted.
- `end` — the ending index (exclusive). Defaults to length of collection if omitted.
- `step` — the amount by which the index increases, defaults to 1. If it's negative, you're slicing over the collection in reverse.

Some common slices:

Given collection, `A`, then

- `A[:]` creates a copy of the entire collection. (uses default value for start, end, and step)
- `A[::-1]` creates a copy of the entire collection in reverse (step=-1 reverses the collection)

Slicing

II

A

0	1	2	3	4	5	6	7	8	9	10
1	2	4	8	16	32	64	128	256	512	1024
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

A[9]

0	1	2	3	4	5	6	7	8	9	10
	1	2	4	8	16	32	64	128	256	512
	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2
										-1

A[9] ↓

1	2	4	8	16
----------	----------	----------	----------	-----------

Create a subsequence from start of sequence, from index (inclusive) **start** (default=0) up to index (but excluding) **end=5**.

Slicing

A

0	1	1	2	2	4	3	8	4	16	5	32	6	64	7	128	8	256	9	512	10	1024
	-11		-10		-9		-8		-7		-6		-5		-4		-3		-2		-1

A[9] ↓

A[:5]

1	2	4	8	16
---	---	---	---	----

A[5:8]

32	64	128
----	----	-----

A[8:]

256	512	1024
-----	-----	------

Create a subsequence *from start of sequence*, from index (inclusive) **start** (default=0) up to index (but excluding) **end=5**.

Create a subsequence *from middle of sequence*, from index (inclusive) **start=5** up to index (but excluding) **end=8**.

Create a subsequence *from end of sequence*, from index (inclusive) **start=8** up to index (but excluding) **end** (default length of sequence=11).

Slicing

A

0	1	2	3	4	5	6	7	8	9	10
	1	2	4	8	16	32	64	128	256	512
	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2
										1024

Diagram showing step increments of +3 from index 0 to 3, and from index 3 to 6. An arrow points to index 9 labeled A[9].

A[:5]

1	2	4	8	16
---	---	---	---	----

Create a subsequence *from start of sequence*, from index (inclusive) **start** (default=0) up to index (but excluding) **end=5**.

A[5:8]

32	64	128
----	----	-----

Create a subsequence *from middle of sequence*, from index (inclusive) **start=5** up to index (but excluding) **end=8**.

A[8:]

256	512	1024
-----	-----	------

Create a subsequence *from end of sequence*, from index (inclusive) **start=8** up to index (but excluding) **end** (default length of sequence=11).

A[0:9:3]

1	8	64
---	---	----

Create a subsequence from index (inclusive) **start=0** up to index (but excluding) **end=9** using a **step=3**.

Iterating over Collections

- Python's **for** is used to iterate over elements in a collections.
- Function **enumerate** counts the elements during iteration.

```
18 ● A = [1,2,4,8,16,32,64,128,256,512,1024]
1
2
3 # loop over all elements
4 for value in A:
5     print(value)
6
7 # count and looping over all elements
8 for pos, value in enumerate(A):
9     print(pos, value)
10
11 # loop over all positions - rarely used in python
12 for pos in range(len(A)):
13     print(pos, A[pos])
```



Iterating over Collections

- Python's **for** is used to iterate over elements in a collections.
- Function **enumerate** counts the elements during iteration.

```
19 ● A = [1,2,4,8,16,32,64,128,256,512,1024]
1
2
3 # loop over all elements
4 ● for value in A:
5 ●   print(value)
6
7 # count and looping over all elements
8 for pos, value in enumerate(A):
9   print(pos, value)
10
11 # loop over all positions - rarely used in py
12 for pos in range(len(A)):
13   print(pos, A[pos])
```

1
2
4
8
16
32
64
128
256
512
1024



Iterating over Collections

- Python's **for** is used to iterate over elements in a collections.
- Function **enumerate** counts the elements during iteration.

```

20
1 A = [1,2,4,8,16,32,64,128,256,512,1024]
2
3 # loop over all elements
4 for value in A:
5     print(value)
6
7 # count and looping over all elements
8 for pos, value in enumerate(A):
9     print(pos, value)
10
11 # loop over all positions - rarely used in py
12 for pos in range(len(A)):
13     print(pos, A[pos])
  
```

1
2
4
8
16
32
64
128
256
512
1024

0 1
1 2
2 4
3 8
4 16
5 32
6 64
7 128
8 256
9 512
10 1024



Filtering

Definition 15 (Filtering)

Build a collection from another by selecting (**filtering**) elements in the collection that satisfy some criteria.

Task — Given a list of powers of 2, select all values that have remainder 4 when divided by 10:

```
21 ●
1  A = [1,2,4,8,16,32,64,128,256,512,1024]
2
3  # old style filtering
4  B = []
5  for value in A:
6      if value % 10==4:  # remainder is 4
7          B.append(value)
8  print(B)
9
10 # or using list comprehension
11 B = [value for value in A if value % 10==4]
12 print(B)
```



Filtering

Definition 15 (Filtering)

Build a collection from another by selecting (**filtering**) elements in the collection that satisfy some criteria.

Task — Given a list of powers of 2, select all values that have remainder 4 when divided by 10:

```
22 ●
1  A = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]
2
3  # old style filtering
4  B = []
5  for value in A:
6      if value % 10 == 4:  # remainder is 4
7          B.append(value)
8  print(B)
9
10 # or using list comprehension
11 B = [value for value in A if value % 10 == 4]
12 print(B)
```

```
[4, 64, 1024]
[4, 64, 1024]
```



Filtering

Definition 15 (Filtering)

Build a collection from another by selecting (**filtering**) elements in the collection that satisfy some criteria.

Task — Given a list of powers of 2, select all values that have remainder 4 when divided by 10:

```

23 ●
1  A = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]
2
3  # old style filtering
4  ● B = []
5  ● for value in A:
6  ●     if value % 10 == 4: # remainder is 4
7  ●         B.append(value)
8  ● print(B)
9
10 # or using list comprehension
11 B = [value for value in A if value % 10 == 4]
12 print(B)

```

Create empty list. Loop over original.
If element satisfies criteria, then append it to list.

[4, 64, 1024]
[4, 64, 1024]



Filtering

Definition 15 (Filtering)

Build a collection from another by selecting (**filtering**) elements in the collection that satisfy some criteria.

Task — Given a list of powers of 2, select all values that have remainder 4 when divided by 10:

24 ●
1 A = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]

2

3 *# old style filtering*

4 B = []

5 **for** value **in** A:

6 **if** value % 10 == 4: *# remainder is 4*

7 B.append(value)

8 **print**(B)

9

10 *# or using list comprehension*

11 ● B = [value **for** value **in** A **if** value % 10 == 4]

12 **print**(B)

Create empty list. Loop over original.
If element satisfies criteria, then append it to list.

List comprehension

[4, 64, 1024]

[4, 64, 1024]



Filtering

Definition 15 (Filtering)

Build a collection from another by selecting (**filtering**) elements in the collection that satisfy some criteria.

Task — Given a list of powers of 2, select all values that have remainder 4 when divided by 10:

25 ●
1 A = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]

2

3 *# old style filtering*

4 B = []

5 **for** value **in** A:

6 **if** value % 10 == 4: *# remainder is 4*

7 B.append(value)

8 **print**(B)

9

10 *# or using list comprehension*

11 B = [value **for** value **in** A **if** value % 10 == 4]

12 **print**(B)

Create empty list. Loop over original.
If element satisfies criteria, then append it to list.

List comprehension

[4, 64, 1024]

[4, 64, 1024]



List comprehension

Definition 16 (List comprehension)

List comprehension is a compact syntax to construct a new sequence from another collection

It consists of

`[EXPRESSION for value in COLLECTION if CONDITION]`

where

- **EXPRESSION** is any python expression.
 - **COLLECTION** is any python collection (set, list, ...)
 - **CONDITION** — is python expression that results in **True** or **False**
-
- As a programmer you don't have to use list comprehensions and instead use the longer traditional style, but you will need to be able to read and understand it since it is the default style in modern Python programmers.
 - Replacing `[` and `]` by `{` and `}` will create a **set** instead of a new **list**.

List Comprehension Example 1

Task — Create list of first 10 square numbers from the set of natural numbers ($\mathbb{N}$).

26 ●

```
1 # traditional approach
2 squares = []
3 for k in range(10):
4     squares.append(k**2)
5 print(squares)
6
7 # using list comprehension
8 squares = [k**2 for k in range(10)]
9 print(squares)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```



List Comprehension Example 1

Task — Create list of first 10 square numbers from the set of natural numbers ($\mathbb{N}$).

27 ●

```
1 # traditional approach
2 squares = []
3 for k in range(10):
4     squares.append(k**2)
5 print(squares)
6
7 # using list comprehension
8 squares = [k**2 for k in range(10)]
9 print(squares)
```

Create empty list. Loop over original collection, calculate expression ($k**2$) and append result to list.

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]



List Comprehension Example 1

Task — Create list of first 10 square numbers from the set of natural numbers ($\mathbb{N}$).

28 ●

```
1 # traditional approach
2 squares = []
3 for k in range(10):
4     squares.append(k**2)
5 print(squares)
```

```
6
7 # using list comprehension
8 ● squares = [k**2 for k in range(10)]
9 print(squares)
```

Create empty list. Loop over original collection, calculate expression ($k**2$) and append result to list.

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]



List Comprehension Example 1

Task — Create list of first 10 square numbers from the set of natural numbers ($\mathbb{N}$).

29 ●

```

1 # traditional approach
2 squares = []
3 for k in range(10):
4     squares.append(k**2)
5 print(squares)
6
7 # using list comprehension
8 squares = [k**2 for k in range(10)]
9 print(squares)

```

Create empty list. Loop over original collection, calculate expression ($k**2$) and append result to list.

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]



- The COLLECTION is `range(10)` which generates the list `[0,1,2,3,4,5,6,7,8,9]`.
- The EXPRESSION is `k**2` which generates the required pattern.
- There is no CONDITION so all elements in COLLECTION are used.

List Comprehension Example 2

Task — Create list of even integers up to but not including 10.

```
30 ●
1  # traditional approach
2  evens = []
3  for k in range(10):
4      if k % 2 == 0:
5          evens.append(k)
6  print(evens)
7
8  # using list comprehension
9  evens = [k for k in range(10) if k % 2 == 0]
10 print(evens)
```

```
[0, 2, 4, 6, 8]
[0, 2, 4, 6, 8]
```



List Comprehension Example 2

Task — Create list of even integers up to but not including 10.

```
31 ●
1  # traditional approach
2  ● evens = []
3  ● for k in range(10):
4  ●     if k % 2 == 0:
5  ●         evens.append(k)
6  ● print(evens)
7
8  # using list comprehension
9  evens = [k for k in range(10) if k % 2 == 0]
10 ● print(evens)
```

Create empty list. Loop over original collection, if value matches criteria (even), then append value to list.

[0, 2, 4, 6, 8]
[0, 2, 4, 6, 8]



List Comprehension Example 2

Task — Create list of even integers up to but not including 10.

```
32 ●  
1  # traditional approach  
2  evens = []  
3  for k in range(10):  
4      if k % 2 == 0:  
5          evens.append(k)  
6  print(evens)  
7  
8  # using list comprehension  
9 ● evens = [k for k in range(10) if k % 2 == 0]  
10 print(evens)
```

Create empty list. Loop over original collection, if value matches criteria (even), then append value to list.

```
[0, 2, 4, 6, 8]  
[0, 2, 4, 6, 8]
```



List Comprehension Example 2

Task — Create list of even integers up to but not including 10.

```

33 1 # traditional approach
2  evens = []
3  for k in range(10):
4      if k % 2 == 0:
5          evens.append(k)
6  print(evens)
7
8  # using list comprehension
9  evens = [k for k in range(10) if k % 2 == 0]
10 print(evens)

```

Create empty list. Loop over original collection, if value matches criteria (even), then append value to list.

```

[0, 2, 4, 6, 8]
[0, 2, 4, 6, 8]

```



- The COLLECTION is `range(10)` which generates the list `[0,1,2,3,4,5,6,7,8,9]`.
- The EXPRESSION is `k` which generates the required pattern.
- The CONDITION, `k%2==0` selects the even integers only.

List Comprehension Example 3

Task — Create list of the length of each word in a list of words.

```
34 ● 1 names = ['Alice', 'Bob', 'Charlie']  
2  
3 # traditional approach  
4 lengths = []  
5 for name in names:  
6     lengths.append(len(name))  
7 print(lengths)  
8  
9 # using list comprehension  
10 lengths = [len(name) for name in names]  
11 print(lengths)
```

```
[5, 3, 7]
```

```
[5, 3, 7]
```



List Comprehension Example 3

Task — Create list of the length of each word in a list of words.

```
35 ● 1 names = ['Alice', 'Bob', 'Charlie']
2
3   # traditional approach
4 ● 4 lengths = []
5 ● 5 for name in names:
6 ● 6     lengths.append(len(name))
7   7 print(lengths)
8
9   # using list comprehension
10 10 lengths = [len(name) for name in names]
11 11 print(lengths)
```

Create empty list. Loop over original collection, calculate length of word, then append result to list.

[5, 3, 7]

[5, 3, 7]



List Comprehension Example 3

Task — Create list of the length of each word in a list of words.

```
36 ● 1 names = ['Alice', 'Bob', 'Charlie']  
2  
3 # traditional approach  
4 lengths = []  
5 for name in names:  
6     lengths.append(len(name))  
7 print(lengths)  
8  
9 # using list comprehension  
10 ● lengths = [len(name) for name in names]  
11 print(lengths)
```

Create empty list. Loop over original collection, calculate length of word, then append result to list.

[5, 3, 7]

[5, 3, 7]



List Comprehension Example 3

Task — Create list of the length of each word in a list of words.

```
37 1 names = ['Alice', 'Bob', 'Charlie']  
2  
3  # traditional approach  
4  lengths = []  
5  for name in names:  
6      lengths.append(len(name))  
7  print(lengths)  
8  
9  # using list comprehension  
10 lengths = [len(name) for name in names]  
11 print(lengths)
```

Create empty list. Loop over original collection, calculate length of word, then append result to list.

[5, 3, 7]

[5, 3, 7]



- The COLLECTION is names, a list of strings.
- The EXPRESSION is `len(name)` which computes the length of the string stored in name.
- There is no CONDITION so all elements in COLLECTION are used.

Aside - Tuples

Definition 17 (Tuple)

A **tuple** is ordered, immutable collection.

- A **immutable** collection is unchangeable, meaning that we cannot change, add or remove items after the collection has been created.
- Tuple are denoted by round brackets, (and).
- Unfortunately, round brackets are also used in controlling the order of operations in expressions. So a tuple with just one element requires a comma.

```
38 ●  
1  fruits = ("apple", "banana", "cherry")  
2  print(fruits)  
3  
4  fruits = ("apple",)  
5  print(fruits)
```

```
('apple', 'banana', 'cherry')  
( 'apple', )
```

Outline

1. Sequences	2
2. Arithmetic and Geometric Progressions	16
2.1. Definition of Arithmetic and Geometric Progression	17
2.2. Partial Sums of AP and GP	19
3. Implementing Sequence Collections in Python	25
3.1. Common Concepts	26
3.2. Lists	27
3.3. Tuples	37
4. Strings	38

Strings

Definition 18 (string `str`)

A **str** is a sequence collection consisting of a sequence of characters, like letters, numbers, and symbols.

Since a `str` is a sequence collection, all of the sequence operations we covered in `lists` also apply to `str`

Slight change in notes — we will come back to this section after functions.